

技术数据 · Optic-SpaceNet 光学 CNN 在轨加速系统

复赛作品 CICC 1003564
仓库结构、完整复现步骤、关键脚本说明、依赖清单与编码规范。
所有命令从仓库根目录发起：脚本内部数据/权重路径（`data/EuroSAT_RGB`、`weights/...`）
均相对仓库根解析。

目录

- 1. [仓库结构概览](#)
- 2. [复现步骤](#)
- 3. [关键脚本说明](#)
- 4. [依赖清单](#)
- 5. [编码规范](#)

1. 仓库结构概览

远程仓库：
[lhh010/Ltsimulator-EuroSAT: 复赛尝试：面向航空航天“在轨计算”的光学 CNN 加速系统 \(Optic-SpaceNet\)](#)
(后面换成gitee的)

```
Ltsimulator-EuroSAT/
├── src/                                # 全部 Python 代码（53 个文件）
│   ├── _pathsetup.py                 # sys.path 引导（入口脚本自动调用，无需手动）
│   ├── core/
│   │   └── optic_layers.py           # 光计算核心库（OpticalEngine / OpticConv2d /
OpticLinear / build_optical_model）
│   ├── qat/                          # 量化感知训练库（迭代版本）
│   │   ├── optic_qat.py              # v1: 初版（fake_int4 / LSQ / BN 融合）
│   │   ├── optic_qat_v2.py           # v2: uint4/int4 非对称、LSQ+（旧版有 bug）
│   │   ├── optic_qat_v3.py           # v3: int8 激活、BN 保留、无 first_layer_fp32
│   │   └── optic_qat_v4.py           # v4: Gazelle 硬件匹配（int8 权重 +
GazelleNoiseInjector + 首层 FP32）★主力
│   └── optic_qat_lsqr.py             # LSQ+ 修复版（int8 可学习 scale/zp + LSQ 梯度）
★对比
├── data/
```

```

|   |   └─ eurosat_split.py      # 单一数据源: 干净三分 split (assert 兜底)
|   └─ training/                  # 模型定义 + 训练入口 + 训练器
|   |   └─ train_phase4_runner.py  # Phase4 训练器 + WarmupCosine +
load_eurosat_data
|   |   └─ train_mixed_runner.py   # Mixed 训练器
|   |   └─ model1_baseline*.py     # Model 1 (FP32 / qat / int4 / mixed /
phase4 / phase4_v3)
|   |   └─ model2_spacenet_v1*.py  # Model 2 (FP32 / qat / int4 / lsq /
mixed / phase4 / v2 / v3) ★
|   |   └─ model3_spacenet_v2*.py  # Model 3 (FP32 / qat / int4 / mixed /
phase4 / v2 / v3 + KD) ★
|   └─ scripts/                   # 容器推理 / 交叉验证 / 出图 / 鲁棒性扫描
|       └─ example_load_gazelle_model.py  # osimulator API 参考
|       └─ optic_inference_int8.py        # Model 2 int8 容器
(Optic/QAT/MOPs) ★
|       └─ optic_inference_int8_model1.py  # Model 1 int8 容器 (变体 A/B) ★
|       └─ optic_inference_kd.py          # Model 3 KD int8 容器 ★
|       └─ optic_inference_lsq.py         # LSQ+ 专用容器
|       └─ optic_inference_int4.py(_v2)   # INT4 容器
|       └─ optic_inference_phase4.py      # Phase4 模型容器
|       └─ optic_inference_mixed(_model1).py # Mixed 模型容器
|       └─ optic_inference_h{1,2,3}.py    # 光电混合提速探索方案
|       └─ noise_robustness.py(_v2).py    # 噪声鲁棒性扫描
|       └─ plot*.py                      # 出图 (compute_vs_accuracy /
accuracyBars / optical_ratio / phase_evolution / six_stage_climb /
eurosat_samples)
└─ weights/                           # 全部 .pth 权重 (25 个, git 跟踪)
└─ docs/                               # 文档 + 图表
    └─ EXPERIMENTS.md                 # 完整实验轨迹 (七阶段 + Bug 记录)
    └─ SUMMARY.md                     # 报告向总结
    └─ PHASE4_DESIGN.md               # Phase4 设计文档
    └─ OPTIC_QAT_README.md            # QAT 技术文档
    └─ TODO.md
    └─ 复赛-test.md                   # 竞赛设计文档
    └─ figures/                       # 图表 (compute_vs_accuracy_final.
{png,pdf,svg} / noise_robustness*.png)
└─ logs/                             # 运行日志 (log.md / log_*.md / *.log)
└─ data/                             # EuroSAT_RGB 数据集 (原地保留, gitignore)
└─ osimulator/                       # Gazelle 光计算仿真器 (外部 vendored 包,
gitignore)
    └─ GAZELLE_ARCHITECTURE.md        # 硬件架构参考
└─ demo/                             # 可视化前端 + 推理服务
    └─ web/index.html                 # 前端页面
    └─ server/                       # 推理服务 (app.py / inference_local.py /
remote_client.py / render.py)
    └─ remote/optic_server.py         # 远程光计算服务端

```

```
|   |— docs/           # API / 设计 / 前端文档
|   |— tests/          # 契约 + 单元测试
|   |— deploy.sh       # 部署脚本
|— 复赛文档/          # 本三份报告
|— README.md          # 仓库说明
|— AGENTS.md          # 远程主机/容器约定
```

2. 复现步骤

2.1 环境准备

```
# 克隆仓库
cd /path/to/train-test

# 创建虚拟环境（本机训练/QAT 评估用）
python -m venv venv
source venv/bin/activate      # Linux/Mac
# venv\Scripts\activate      # Windows

# 安装本机依赖
pip install torch torchvision numpy matplotlib
```

光计算真机评估需进入主办方提供的 Docker 容器（见 §4.2）。容器外（如 Windows 本机）`import osimulator` 会失败并回退到 `FakeOpticalEngine`，只能跑 QAT 伪量化（秒级）与 NumPy 路径。

2.2 数据准备

先下载数据集：[卫星图像多分类数据集 · 数据集](#)

EuroSAT RGB 数据集放入 `data/EuroSAT_RGB/`（`ImageFolder` 结构，10 个子目录对应 10 类）。全集 27000 张、64×64。

数据划分由 `src/data/eurosat_split.py` 统一管理（`seed=42`，`val_ratio=test_ratio=0.2` → `train 16200 / val 5400 / test 5400`，三者严格不相交）。自检：

```
python src/data/eurosat_split.py
# 输出: n=27000 | train=16200 val=5400 test=5400; 互斥+覆盖自检通过 [OK]
```

2.3 模型训练（int8 部署配方，从零 QAT）

```
# Model 1 Phase4 v3 int8（变体 A，默认；--variant B 走变体 B）
python src/training/model1_baseline_phase4_v3.py --variant A      # ~4.7h（CPU）

# Model 2 Phase4 v3 int8（SpaceNet V1, stem FP32 + Gazelle 噪声）
```

```
python src/training/model2_spacenet_v1_phase4_v3.py # ~72.8 min

# Model 3 Phase4 v3 int8+KD (SpaceNet V2, 叠加 ResNet-18 蒸馏)
python src/training/model3_spacenet_v2_phase4_v3.py # ~90.4 min
```

训练完成后权重保存至 `weights/` (如 `spacenet_v1_phase4_v3_int8.pth`)。FP32 基准模型可用 `model1_baseline.py` / `model2_spacenet_v1.py` / `model3_spacenet_v2.py` 训练。

2.4 精度评估 (QAT 伪量化交叉验证, 秒级, 容器外可用)

```
# Model 2 int8: QAT 伪量化全量 test
python src/scripts/optic_inference_int8.py --qat --batch 256
# 期望: Int8 QAT test ≈ 92.20% (与 val 92.06% 一致, 无泄漏)

# Model 1 变体 A int8
python src/scripts/optic_inference_int8_model1.py --variant A --qat --batch 256
# 期望: Int8 QAT test ≈ 97.89%

# Model 3 KD (注: --qat 路径走 int4, int8 数只能走 Optic/osim 模式)
python src/scripts/optic_inference_kd.py --qat --batch 256
```

2.5 模拟器评估 (osimulator 真实光计算硬件仿真, 容器内)

```
# 切换 Docker context 并进入容器
docker context use fdusc-cpu-135
docker exec -it gazelle_sim bash
cd /workspace/train-test

# Model 2 全量 5400 (~3.7h) — 当前真机真值
/local/miniconda/envs/moca_llm/bin/python src/scripts/optic_inference_int8.py
# 期望: 光计算准确率 ≈ 90.43% (全量 5400)

# Model 3 全量 5400 (~3.7h)
/local/miniconda/envs/moca_llm/bin/python src/scripts/optic_inference_kd.py
# 期望: 光计算准确率 ≈ 90.28% (全量 5400)

# Model 1 抽样 (全量 ~9 天不可行)
/local/miniconda/envs/moca_llm/bin/python
src/scripts/optic_inference_int8_model1.py --variant A --quick 650
# 期望: ≈ 98.15% (q650)

# 快速抽样验证 (~3min)
```

```
/local/miniconda/envs/moca_llm/bin/python src/scripts/optic_inference_int8.py
--quick 50
```

推理默认 `--batch 1` (osimulator 安全值), 可用 `--batch N` 调整。Optic 模式 (默认) 走真实 osimulator; `--qat` 走 PyTorch 伪量化交叉验证; `--mops-only` 仅统计光计算占比。

2.6 噪声鲁棒性测试

```
python src/scripts/noise_robustness_v2.py --all
# 扫描: 权重量化位宽 / 权重高斯噪声 / 激活高斯噪声 / 激活量化位宽 / 权重 dropout
# 结果保存为 noise_robustness_v2.png + 控制台汇总表
```

2.7 光计算占比 / MOPs 统计

```
python src/scripts/optic_inference_int8.py --mops-only
# 输出: 每层类型 / 展平长度 / 对齐率 / MOPs / 计算位置 (光电分布)
#      + 整机光计算占比 (Model 2/3 = 90.65%)
```

2.8 出图

```
python src/scripts/plot_compute_vs_accuracy.py      # 计算量-精度图 → docs/figures/
python src/scripts/plot_accuracyBars.py             # 精度柱状对比
python src/scripts/plot_optical_ratio.py            # 光计算占比
python src/scripts/plot_phase_evolution.py          # 七阶段精度演进
python src/scripts/plot_six_stage_climb.py          # 六阶段爬升曲线
python src/scripts/plot_eurosat_samples.py          # EuroSAT 样本可视化
```

2.9 可视化前端 / 推理服务 (demo)

```
bash demo/deploy.sh                                # 部署
# 本地推理服务
python demo/server/app.py                          # Web 后端 (含 inference_local.py 本地推理)
# 远程光计算服务 (连接容器)
python demo/remote/optic_server.py
# 前端: demo/web/index.html
```

3. 关键脚本说明

3.1 训练脚本 (src/training/)

脚本	用途	参数/说明
<code>model1_baseline_phase4_v3.py</code>	Model 1 Phase4 v3 int8 训练	<code>--variant A</code> (默认, conv1_1 电计算, 光占比 97.7%) / <code>--variant B</code> (+conv3_2 电计算, 73.6%)
<code>model2_spacenet_v1_phase4_v3.py</code>	Model 2 Phase4 v3 int8 训练	<code>--wbits 8</code> (默认) / <code>--wbits 4</code> (int4 对比); stem FP32 + Gazelle 噪声
<code>model3_spacenet_v2_phase4_v3.py</code>	Model 3 Phase4 v3 int8+KD 训练	<code>--wbits 8</code> (默认); ResNet-18 教师, T=4.0、 $\alpha=0.7$
<code>model1_baseline.py</code> / <code>model2_spacenet_v1.py</code> / <code>model3_spacenet_v2.py</code>	FP32 基准训练	分别产出 97.17% / 90.15% / 91.44% 基准
<code>model*_phase4_v2.py</code>	Phase4 v2 int4 (QAT 全开修复版)	对照 int4 路线
<code>model*_mixed.py</code>	Mixed 精度 (Conv=int4 光 / Linear=fp32 电)	Model 1 Mixed 98.26%
<code>train_phase4_runner.py</code>	Phase4 训练器 + WarmupCosineScheduler + load_eurosat_data	三个模型共用训练管线
<code>train_mixed_runner.py</code>	Mixed 训练器 (MixedPrecisionTrainer)	—

3.2 核心库 (`src/core/` 、 `src/qat/`)

文件	用途
<code>optic_layers.py</code>	光计算推理核心 : OpticalEngine (真实/模拟双引擎自动检测)、OpticConv2d (im2col→光学矩阵乘→col2im)、OpticLinear、build_optical_model (模型转换工厂, keep_first_conv_electronic)、噪声注入器 (Gaussian/Phase/Shot/Crosstalk)、对齐率计算

文件	用途
optic_qat_v4.py	主力 QAT: fake_quantize_symmetric (STE)、QATConv2d_v4 / QATLinear_v4 (int8 权重 + per-channel)、GazelleNoiseInjector (DAC ENOB 7.5 + TIA $\sigma=5.34e-4$)、prepare_model_v4 (首层 FP32)、print_alignment_detail
optic_qat_lsq.py	LSQ+ 对比 QAT: _LSQPlusFn (自定义 autograd, STE + LSQ 梯度 $1/\sqrt{N \cdot n_levels}$)、LSQConv2d / LSQLinear (可学习 scale/zp)、prepare_model_lsq、set_lsq_lr (0.1× 独立学习率)
optic_qat_v3.py	v3: int8 激活、BN 保留、无 first_layer_fp32 (int4 路线用)
eurosat_split.py	单一数据源: 三分 split + assert 互斥/覆盖不变量 (杜绝 Bug #11 泄漏)

3.3 容器推理脚本 (src/scripts/)

脚本	模型	模式	关键产出
optic_inference_int8.py	Model 2 int8	Optic (默认) / --qat / --mops-only	真机全量 90.43%、MOPs 占比 90.65%
optic_inference_int8_model1.py	Model 1 int8	--variant A/B、--quick N	q650 真机 A 98.15% / B 97.54%
optic_inference_kd.py	Model 3 KD int8	Optic (默认) / --qat	真机全量 90.28%
optic_inference_lsq.py	LSQ+ int8	LSQ 专用路径 (monkey-patch 保留学习到的 scale/zp)	LSQ+ 真机验证
optic_inference_int4.py (_v2)	Model 2 int4	Optic / --qat	int4 路线 ~88% 上限 (边界说明)
optic_inference_mixed(_model1).py	Mixed	QAT / Optic	Model 1 Mixed 抽样 100%
optic_inference_h{1,2,3}.py	光电混合提速探索	Optic	部分层切回电计算换吞吐
noise_robustness_v2.py	三模型 int4	鲁棒性扫描	5 类扰动 × 6 级, val 集 5400

脚本	模型	模式	关键产出
<code>example_load_gazelle_model.py</code>	—	API 参考	<code>osimulator</code> <code>load_gazelle_model</code> 调用范例

4. 依赖清单

4.1 本机依赖

包	用途
<code>torch</code>	深度学习框架，训练与 QAT
<code>torchvision</code>	EuroSAT 数据加载、ResNet-18 教师模型
<code>numpy</code>	推理引擎、数据 IO、MOPs 统计
<code>matplotlib</code>	可视化（精度对比、计算量-精度、鲁棒性图）

安装：

```
pip install torch torchvision numpy matplotlib
```

4.2 光子模拟器（容器内置，不可 pip 安装）

包	版本	说明
<code>osimulator</code>	1.3.4	Gazelle 光计算仿真器 API，容器内置 (<code>lightelligence.docker:lt-simulator_v1.4.6</code>)
<code>entrance</code>	C 扩展	光子硬件入口，容器内置
<code>numpy</code>	1.22.4	容器内 <code>conda env moca_llm</code>

`osimulator` 与 `entrance` 仅存在于 `lightelligence.docker:lt-simulator_v1.4.6` 容器中，不可通过 pip 安装。容器内必须使用 `/local/miniconda/envs/moca_llm/bin/python`（默认 Python 找不到 `osimulator`）。

4.3 硬件/算力环境

- 远程主机 `fdusc-cpu-135`：256 核 Xeon 6980P，377GB RAM；
- 单个 `osimulator` 大 GEMM（如 3136×756×64）耗时约 80s，占用约 90 个 CPU core；
- Model 1（VGG）在光模拟器上极慢（约 150s/图）；Model 2/3 约 2.5s/图。

5. 编码规范

本项目遵循以下工程规范：

1. **分层组织**：核心库（`core/`）、量化库（`qat/`）、数据（`data/`）、训练（`training/`）、脚本（`scripts/`）职责分离，避免循环依赖。
2. **单一数据源**：所有 `train/val/test` 划分必须经 `eurosat_split.py` 获取，`assert` 强制三者互斥且覆盖全集，从源头杜绝数据泄漏（Bug #11）。
3. **从仓库根运行**：所有入口脚本顶部有 3 行 `sys.path` 引导（调用 `src/_pathsetup.py`），把 `src/` 各子目录与仓库根加入 `sys.path`，代码内扁平 `import`（`from optic_qat_v4 import ...`、`from optic_layers import ...`、`import osimulator`）无需修改即可工作；数据/权重路径（`data/EuroSAT_RGB`、`weights/...`）相对仓库根解析。
4. **训练 ↔ 推理配置对齐**：严格保持 `first_conv_fp32`（训练）↔ `keep_first_conv_electronic`（推理）、`weight_bits=8`（训练）↔ `weight_bit=8`（推理）、`Gazelle` 噪声（训练）↔ `osim` 物理噪声（推理）三项对齐（详见《设计报告》§6.4），这是真机近无损的工程纪律。
5. **双引擎自动检测**：`OpticalEngine` 自动检测 `osimulator` 可用性，容器外优雅回退 `FakeOpticalEngine`（打印 `WARN`），保证同一份代码在训练机/容器内均可运行。
6. **科学诚信**：日志全量保留（`logs/`），作废数据（leaky `osim`）明确标注，抽样数与全量数严格区分，精度演进与 Bug 记录完整可追溯（`docs/EXPERIMENTS.md`）。
7. **模块自测**：核心库（`optic_layers.py`、`optic_qat_v4.py`、`optic_qat_lsqr.py`、`eurosat_split.py`）均含 `__main__` 自测，可独立运行验证。

完整实验轨迹见 `docs/EXPERIMENTS.md`，总结见 `docs/SUMMARY.md`，原始日志见 `logs/`，硬件架构参考见 `osimulator/GAZELLE_ARCHITECTURE.md`。